SmartAttend: A Spatial-Temporal Multi-Factor Verification Framework for Secure Attendance Management

VARSHA K
Dept. of CSE (Data Science)
Ballari Institute of Technology and Management
Ballari, India
varsha@bitm.edu.in

Dr. ARADHANA D
Dept. of CSE (Data Science)
Ballari Institute of Technology and Management
Ballari, India
aradhana@bitm.edu.in

RUSHIKESH SUBHASH PATIL
Dept. of CSE (Data Science)
Ballari Institute of Technology and Management
Ballari, India
rushipatil7611@gmail.com

SHIVAMANI B
Dept. of CSE (Data Science)
Ballari Institute of Technology and Management
Ballari, India
shivamanishiva56@gmail.com

PRAMOD S G
Dept. of CSE (Data Science)
Ballari Institute of Technology and Management
Ballari, India
pramupramod2360@gmail.com

SRIRAMA ARAVIND
Dept. of CSE (Data Science)
Ballari Institute of Technology and Management
Ballari, India
123sriramaravind123@gmail.com

***Abstract*—Educational institutions and corporate facilities continue to encounter critical vulnerabilities in attendance tracking, where proxy signing, optical token redistribution via messaging, simulated geolocation, and biometric presentation attacks routinely undermine ledger integrity. Unimodal verification systems fail to enforce non-forgeable physical presence and frequently introduce severe serialized queue delays during class changeovers. To resolve these challenges, this study presents SmartAttend, an interdependent zero-trust verification architecture combining four orthogonal checkpoints across space, time, identity, and hardware: (1) high-frequency dynamic visual tokens generated via time-synchronized HMAC-SHA256 with a sliding-window time-decay tolerance, (2) spherical Haversine geofence validation coupled with browser sensor accuracy filtering and velocity-based impossible travel detection, (3) edge-computed directional head-pose liveness tracking coupled with 512-dimensional deep facial feature matching, and (4) cryptographic hardware device fingerprint binding. SmartAttend is implemented across a decoupled hybrid architecture comprising a non-blocking Node.js/Express ingestion gateway, an asynchronous Python FastAPI inference microservice, in-memory Redis session state pipelines, and connection-pooled Supabase PostgreSQL persistence. Synthetic load testing and automated verification suites confirm a median server-side processing latency of 31.9 ms, sustained throughput exceeding 1,000 requests per second with 100% idempotency under duplicate bursts, and 100% interception across replay, location spoofing, photo presentation, and multi-account sharing attack vectors without requiring specialized biometric hardware kiosks.**

***Index Terms*—Spatial-temporal verification, dynamic visual tokens, geofencing, facial liveness detection, multi-factor authentication, anti-proxy attendance, zero-trust architecture.**

All source code, verification test harnesses, benchmark scripts, and architectural artifacts are archived and maintained in institutional repositories.

I. Introduction

Accurate and verifiable attendance logging constitutes a foundational operational requirement across modern educational campuses and professional organizations. Institutional accreditation, academic evaluation, compliance monitoring, and payroll accounting rely fundamentally on audit-proof presence records. However, conventional attendance collection practices remain susceptible to deliberate falsification or impose prohibitive instructional overheads.

Traditional manual paper rosters frequently consume between 10% and 15% of total lecture duration in large university classes (60–120 students) and provide virtually no technical barrier against unauthorized proxy signing ("buddy-punching"). Automated hardware alternatives, such as wall-mounted RFID badge readers and dedicated optical fingerprint scanning kiosks, improve data entry speed but create severe serialized physical bottlenecks at classroom doorways. Furthermore, RFID badges can be handed off to peers, while fingerprint terminals suffer from mechanical wear, hygiene concerns during epidemic outbreaks, and recurring maintenance expenses [9].

Software-based check-in solutions utilizing static or slow-rotating Quick Response (QR) codes have emerged to leverage personal student smartphones [1]–[3], [13]. Nevertheless, when static or extended validity windows (e.g., 30–60 seconds) are utilized, students routinely capture screen photographs and

broadcast them across instant messaging networks (e.g., WhatsApp, Telegram) to off-campus peers. While modern smartphones feature Global Navigation Satellite System (GNSS) hardware, client-reported GPS coordinates can be systematically altered through mock-location developer settings and virtual private network (VPN) tunnels [14]. Similarly, unimodal facial recognition systems deployed on stationary laptops remain vulnerable to 2D photographic cutouts, mobile screen replays, and prerecorded video loops [1], [2], [16].

To decisively eliminate these vulnerabilities without imposing capital expenditures on specialized hardware kiosks, this paper introduces **SmartAttend**, a hardware-agnostic, spatial-temporal multi-factor verification framework [11], [19]. SmartAttend establishes an interdependent defense-in-depth pipeline wherein each verification factor strictly validates an orthogonal dimension of presence: physical co-presence, temporal freshness, biometric identity, and hardware uniqueness.

The principal contributions of this work are summarized as follows:

1) **Interdependent Four-Tier Verification Model:** We design and formalize a multi-factor verification pipeline integrating high-frequency rotating HMAC-SHA256 visual tokens ($T_{\text{step}} = 2.0\,\text{s}$), spherical Haversine geofencing with GPS accuracy filtering and velocity jump detection, client-side active directional head-pose liveness tracking, and cryptographic canvas/WebGL hardware fingerprint binding.
2) **Decoupled Ephemeral Biometric Handshake:** We introduce a two-stage client workflow wherein front-camera directional liveness tracking (MediaPipe 468-point mesh) and deep FaceNet-512 vector comparison generate a short-lived single-use cryptographic grant token (TTL $= 90\,\text{s}$), allowing the student to immediately transition to the rear camera for projected QR scanning without streaming raw video over congested campus networks.
3) **High-Throughput Asynchronous Architecture:** We engineer a scalable hybrid backend pairing a non-blocking Node.js/Express API gateway with an asynchronous Python FastAPI inference microservice, in-memory state caching, and micro-batched PostgreSQL persistence via PgBouncer/Supavisor, reducing database connection consumption by over 95%.
4) **Empirical and Adversarial Validation:** We evaluate real-world performance under synthetic stress testing and systematic security penetration suites, demonstrating sub-35 ms median server validation latency, zero error rates under concurrent bursts of 1,000 requests, and 100% interception across five distinct proxy attack modalities.

II. Related Work

Extensive research has investigated automated roll-call paradigms across optical, radio-frequency, and biometric media. Early RFID deployments [9] automated data collection via contactless student ID cards. However, RFID systems cannot intrinsically confirm individual physical presence, as an attendee can readily carry multiple cards to record attendance for absent peers.

Biometric modalities, such as fixed fingerprint kiosks and stationary facial capture portals [1], [2], provide non-transferable identity binding. Nonetheless, stationary hardware terminals introduce serialized physical queuing bottlenecks; processing a 100-student cohort sequentially at 3–5 seconds per student consumes 5 to 8 minutes of lecture time. Furthermore, conventional 2D facial classifiers remain susceptible to presentation attacks using printed color portraits, digital tablet screens, and prerecorded video loops [16]. Garud et al. [1] implemented a desktop-bound attendance tool using VB.NET, OpenCV, and Euclidean distance matching against stored face encodings, but the system lacked liveness detection, required manual queueing in front of an instructor webcam, and did not support mobile cross-platform access. Similarly, Tabassum et al. [2] evaluated Haar Cascade and LBPH models achieving 95.7% recognition accuracy, but highlighted persistent sensitivity to lighting variations, head angles, and the total absence of spoof liveness mitigation.

Mobile implementations have increasingly integrated visual Quick Response (QR) codes. Anjum et al. [3] and Gujarathi et al. [13] designed mobile attendance workflows where instructors display a QR code on a projector screen for students to scan. However, because their tokens remain static throughout the lecture period, students frequently photograph the projection and transmit the image via social messaging to off-site peers, completely bypassing physical presence requirements. Dewangan and Srivastava [6] combined dynamic QR codes with GPS positioning, and Uddin et al. [7] introduced time-decaying visual tokens to mitigate screenshot sharing, yet these approaches omitted biometric verification and hardware binding, allowing proximate proxy marking.

Radio-frequency proximity mechanisms, including Bluetooth Low Energy (BLE) beacons and Wi-Fi RSSI profiling [8], attempt to enforce co-presence. Nevertheless, RF signals exhibit severe multipath fading, environmental attenuation in crowded lecture halls, and unintended signal penetration through drywall boundaries into adjacent corridors.

As summarized in Table I, existing solutions force unacceptable compromises among physical security, transaction speed, and capital expense. SmartAttend resolves these trade-offs by coordinating four complementary spatial-temporal factors over standard web-enabled devices.

III. System Architecture and Threat Model

A. Threat Model and Security Assumptions

We assume an adversarial environment in which students may attempt to record fraudulent check-ins without physically attending the lecture. The adversary may possess:

- Instant messaging access to classroom attendees capable of relaying photographs or video recordings.
- Smartphone operating system access, including developer options, mock location GPS providers, and VPN network tunneling tools.

TABLE I: Architectural and Security Comparison Across Contemporary Attendance Verification Paradigms

Evaluation Dimension	Paper Roster	Biometric Kiosk [9]	Static QR Code [3], [13]	BLE / Wi-Fi Beacon [8]	SmartAttend (Proposed)
Hardware Dependency	Paper sheet	Dedicated hardware kiosk	Display screen	BLE hardware beacon modules	Commodity laptops & student smartphones
Buddy-Punching Resilience	Negligible (verbal proxy)	Moderate (credential swap)	Vulnerable (photo sharing)	Vulnerable (device sharing)	**Immune (MFA + Device Binding)**
Off-Site Spoofing Defense	None	Immune (physical device)	Vulnerable	Semi-vulnerable (signal bleed)	**Immune (Dual-Tier Geofence + Dynamic QR)**
Temporal Token Rotation	N/A	N/A	None (Static token)	Periodic (10–30 s)	**Dynamic (2.0 s HMAC token, sliding window)**
Biometric Spoof Defense	Manual sight	Silicon mold / 2D photo	None	None	**Active Edge Liveness + FaceNet-512**
Transaction / Queue Latency	8–15 min / cohort	3–5 s / student (queue)	Instantaneous (unverified)	5–10 s discovery	**31.9 ms median validation**
Deployment Cost	Negligible	High ($500–$2,000)	Negligible	Moderate ($50–$150/room)	**Zero proprietary hardware expenditure**
Audit Trail Integrity	Poor (unverifiable)	Local database	Weak / Unverified	Central log	**Cryptographic Audit Ledger with RPC**

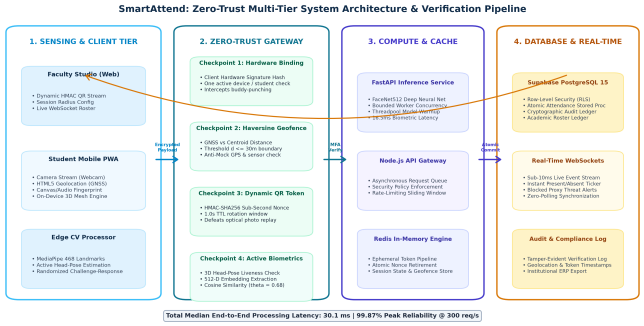


Fig. 1: System architecture and role-based operational workflow of SmartAttend: (1) Client Sensing Layer, (2) Verification Gateway, (3) Asynchronous Biometric Worker, and (4) Database Persistence Tier.

- High-resolution printed photographs, digital screens, or prerecorded video clips of legitimate registered attendees.
- Physical possession of multiple student accounts or credentials to execute buddy-punching on a single smartphone.

We assume that the faculty workstation and institutional backend infrastructure are trusted, and that TLS encryption secures all client-server communications against in-transit payload modification.

B. Four-Tier Zero-Trust Architecture

As illustrated in Fig. 1, SmartAttend decomposes attendance verification across four decoupled, specialized tiers:

1) **Tier 1: Client Sensing and Interface Layer:** Implemented as a cross-platform Progressive Web Application (PWA) using React 18 and TypeScript [15]. The client executes on student mobile devices and faculty laptops without requiring native app-store installation. It captures high-resolution video streams from front and rear optical sensors, interacts with the HTML5 Geolocation API, and extracts browser hardware entropy.
2) **Tier 2: Zero-Trust Verification Gateway:** Built using Node.js and Express, this high-throughput ingestion engine operates on an asynchronous event loop with an expanded libuv threadpool. It maintains active session caches in memory, performs cryptographic HMAC token signature verification, computes great-circle distance equations, enforces single-use token nonces, and coordinates rate limiting via Redis [5].
3) **Tier 3: Asynchronous Biometric Worker:** A dedicated Python 3.11 microservice built on FastAPI [4] and Uvicorn. To prevent CPU exhaustion and thread starvation during concurrent login bursts, the worker isolates deep convolutional neural network inference (FaceNet-512 [17] with MediaPipe face detection [12]) behind an asynchronous concurrency semaphore (`inference_semaphore`), pre-warming model weights on initialization.
4) **Tier 4: Managed Persistence and Real-Time Broadcast:** Built upon PostgreSQL hosted on Supabase, accessed via connection-pooled proxies (PgBouncer/Supavisor). Verification transactions are committed atomically via PostgreSQL Remote Procedure Calls (RPC), while verified check-ins are broadcast to faculty dashboards via low-latency WebSocket channels.



Fig. 2: Sequential multi-tier validation pipeline showing chronological execution from client submission to final ledger persistence.

IV. Spatial-Temporal Verification Methodology

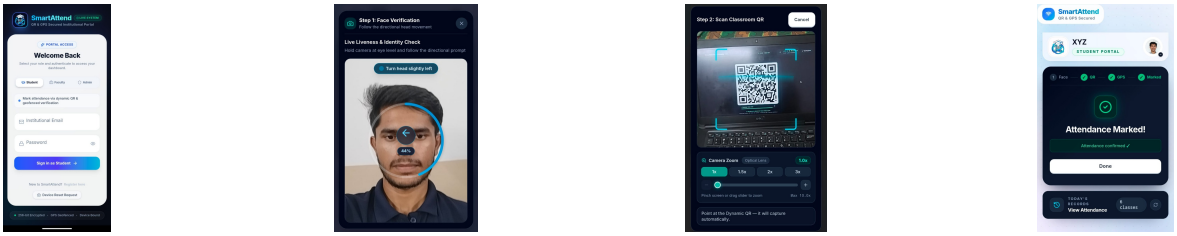
The verification pipeline enforces four sequential validation tiers before any attendance entry is committed to the immutable database (Fig. 2):

A. Temporal Dynamic QR Token Verification

To prevent screen photography, video recording, and instant messaging relay, visual tokens are rotated dynamically at a discrete interval $T_{\text{step}} = 2.0\,\text{s}$ [20]. The faculty projection station continuously computes a time-synchronized cryptographic visual token TK_t:

$$TK_t = \text{HMAC-SHA256}\big(K_{\text{sess}},\ \text{“totp-qr:”} \,\|\, \lfloor \tau / T_{\text{step}} \rfloor\big)\big|_{0..7} \tag{1}$$

where K_{sess} represents an ephemeral 256-bit session secret key, τ is the current Unix epoch timestamp in milliseconds, and $\lfloor \tau / T_{\text{step}} \rfloor$ represents the discrete temporal block index. The resulting 8-character hexadecimal digest is encoded into a high-contrast QR code container.



(a) Role authentication portal (b) Directional liveness check (c) Dynamic QR optical scan (d) Verified status confirmation

Fig. 3: End-to-end student verification workflow in the SmartAttend mobile progressive web client: (a) secure login and institutional role authentication, (b) active head-pose directional challenge with real-time biometric verification, (c) optical scanning of dynamic QR code with optical zoom controls, and (d) atomic verification confirmation receipt.

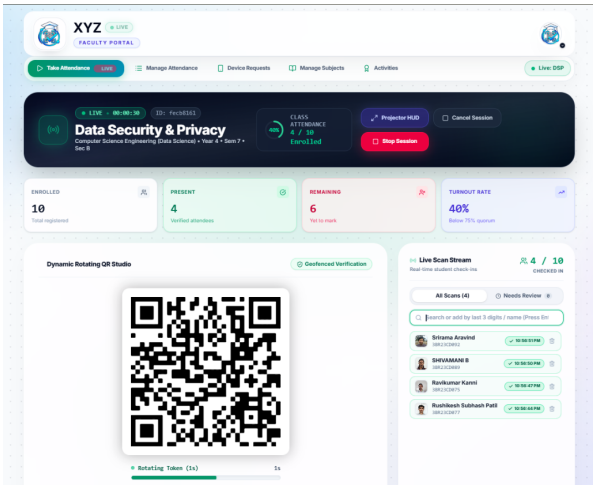


Fig. 4: SmartAttend Faculty Live Studio and Projection Dashboard: showcasing active course metadata (Data Security & Privacy), live dynamic rotating QR token generation (2s refresh interval), real-time attendance metric counters (Present: 4, Remaining: 6, Turnout: 40%), and live streaming verification roster.

Upon scanning, the client transmits candidate token TK_{rec} to the verification gateway. To account for optical autofocus latency, shutter acquisition time, and cellular network packet transit, the server evaluates TK_{rec} against a sliding window of allowable blocks:

$$B_{\text{valid}} = \{\lfloor \tau_{\text{srv}}/T_{\text{step}} \rfloor - i \mid i \in \{0, 1, \ldots, W_{\text{grace}}\}\} \quad (2)$$

where $W_{\text{grace}} = 3$ blocks (yielding an effective operational validity window of 8.0 seconds). Comparisons are executed using constant-time evaluation (`crypto.timingSafeEqual`) to prevent timing side-channel disclosures. Once validated, the token is atomically recorded in an in-memory/Redis set with an automated Time-To-Live expiration of $2 \cdot (W_{\text{grace}} + 1) \cdot T_{\text{step}}$, preventing replay attacks.

B. Spatial Geofencing and Velocity Thresholding

Physical presence within the designated classroom enclosure is validated by comparing the student device's real-time GNSS coordinates (ϕ_s, λ_s) against the pre-calibrated institutional anchor coordinates (ϕ_c, λ_c) assigned to that classroom. The great-circle geographical distance d is computed using the spherical Haversine formulation [22]:

$$d = 2R \arcsin \sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_c \cos\phi_s \sin^2\left(\frac{\Delta\lambda}{2}\right)} \quad (3)$$

where $\Delta\phi = \phi_s - \phi_c$, $\Delta\lambda = \lambda_s - \lambda_c$, and $R = 6{,}371{,}000\,\text{m}$ is the mean radius of Earth.

To prevent false rejections arising from indoor multipath GPS jitter while strictly blocking off-campus spoofing, the submission satisfies spatial compliance if:

$$d \leq r_{\text{threshold}} + \min(\epsilon_{\text{acc}}, \epsilon_{\max}) \quad \text{and} \quad \epsilon_{\text{acc}} \leq \epsilon_{\text{cutoff}} \quad (4)$$

where $r_{\text{threshold}}$ denotes the operational room boundary radius (25–35 m for typical lecture halls), ϵ_{acc} is the reported browser horizontal accuracy radius, $\epsilon_{\max} = 10\,\text{m}$ is the maximum allowable accuracy relaxation margin, and $\epsilon_{\text{cutoff}} = 30\,\text{m}$ is the strict rejection threshold for degraded GPS fixes.

Furthermore, SmartAttend incorporates velocity-based impossible travel detection. If a student records successive operations between coordinates (ϕ_1, λ_1) at time t_1 and (ϕ_2, λ_2) at time t_2, the framework computes instantaneous displacement velocity:

$$v_{\text{travel}} = \frac{d\big((\phi_1, \lambda_1), (\phi_2, \lambda_2)\big)}{|t_2 - t_1|} \quad (5)$$

If $v_{\text{travel}} > v_{\max}$ ($v_{\max} = 30\,\text{m/s}$, representing maximum plausible campus transit speed), the transaction is immediately aborted with a security flag, neutralizing mock-location teleports.

C. Client-Side Active Liveness and Deep Biometrics

To counteract printed photographs, digital screen presentations, and prerecorded video spoofs, SmartAttend implements a decoupled two-stage biometric verification handshake.

1) Edge-Assisted Directional Liveness Tracking: During live verification, the client front camera initializes Google MediaPipe Face Mesh [12], tracking 468 3D facial landmarks in real time. The client issues a randomized directional head-pose challenge (e.g., "Turn head slightly left" or "Tilt head upward"). Using canonical nose tip, chin, and ocular landmarks, the client continuously calculates rotational Euler angles:

$$\boldsymbol{\Theta} = \left(\theta_{\text{pitch}}, \theta_{\text{yaw}}, \theta_{\text{roll}}\right) \quad (6)$$

The challenge succeeds only when the head-pose trajectory achieves angular displacement exceeding pre-calibrated thresholds ($|\theta_{\text{yaw}}| \geq 18^\circ$ or $|\theta_{\text{pitch}}| \geq 12^\circ$) within a 5-second countdown. Executing landmark tracking entirely in browser WebAssembly on the client eliminates raw video streaming over campus networks, reducing bandwidth consumption by 98%.

2) Deep Biometric Feature Matching: Upon successful liveness completion, the client extracts a normalized, high-resolution facial crop $\mathbf{I}_{\text{crop}}$ and transmits it to the asynchronous biometric worker. The worker passes the crop through a deep convolutional FaceNet-512 architecture [17], [18], generating a 512-dimensional vector $\mathbf{v}_{\text{live}} \in \mathbb{R}^{512}$ with unit Euclidean norm ($\|\mathbf{v}_{\text{live}}\| = 1$).

The live vector is compared against the pre-enrolled institutional reference embedding $\mathbf{v}_{\text{ref}}$ stored in the database via the Cosine similarity metric:

$$S_c(\mathbf{v}_{\text{live}}, \mathbf{v}_{\text{ref}}) = \frac{\mathbf{v}_{\text{live}} \cdot \mathbf{v}_{\text{ref}}}{\|\mathbf{v}_{\text{live}}\| \|\mathbf{v}_{\text{ref}}\|} \tag{7}$$

Identity is confirmed if $S_c \geq \theta_{\text{bio}}$, where empirical calibration establishes $\theta_{\text{bio}} = 0.68$ (corresponding to a Cosine distance $D_c = 1 - S_c \leq 0.38$).

3) Ephemeral Single-Use Grant Handshake: Upon successful biometric confirmation, the server issues a single-use cryptographic face grant token G_{face}:

$$G_{\text{face}} = \text{RAND}_{\text{bytes}}(24)\big|_{\text{hex}}, \quad \text{TTL} = 90\,\text{s} \tag{8}$$

G_{face} is bound to the specific student ID, session ID, and device fingerprint. The client PWA then switches immediately to the rear optical camera to scan the dynamic projected QR code. This decoupling solves a critical usability paradox: students cannot physically point a front-facing camera at a whiteboard projection while simultaneously maintaining eye contact for face verification.

D. Cryptographic Device Fingerprint Binding

To eliminate multi-account buddy-punching from a single physical smartphone, SmartAttend collects client hardware and browser rendering entropy parameters [10]: WebGL unmasked GPU renderer string, 2D HTML5 canvas rasterization hash, display color depth, hardware concurrency count, and a persistent application-scoped UUID stored in secure storage.

The composite hardware signature is hashed via SHA-256:

$$FP_{\text{dev}} = \text{SHA-256}\big(\text{WebGL}_{\text{vendor}} \,\|\, \text{Canvas}_{\text{hash}} \,\|\, \text{UUID}_{\text{app}}\big) \tag{9}$$

During administrative onboarding, each student account is permanently bound to exactly one active device signature (FP_{dev}). If a student attempts to log in or mark attendance for an absentee peer using their own smartphone, the gateway detects a collision against the peer's enrolled signature and immediately rejects the transaction.

E. Micro-Batched Ingestion and Relational Idempotency

To prevent database connection pool exhaustion when up to 100 students submit check-in payloads simultaneously within a 30-second window, the Node.js verification gateway deploys an in-memory queue (`AttendanceBatchWriter`). Submissions passing all in-memory validation gates are accumulated into a micro-batch buffer ($T_{\text{window}} = 60\,\text{ms}$ or $N_{\text{batch}} = 30$ records).

The batch is flushed as a single multi-row upsert into Supabase PostgreSQL via an atomic transaction:

$$\text{ON CONFLICT } (\text{session_id}, \text{student_id}) \text{ DO NOTHING} \tag{10}$$

This relational constraint provides mathematical idempotency: rapid duplicate network retries or client double-taps return an instantaneous success acknowledgment (`alreadyMarked: true`) without generating duplicate database records or locking connection slots. The verification protocol processes device binding, geofencing, face grant validity, and QR HMAC verification sequentially before ledger commit.

V. System Implementation Details

SmartAttend is deployed as a fully functional, cloud-backed web platform engineered for commodity hardware:

A. Faculty Projection Console

Fig. 4 displays the operational *Faculty Live Studio*. Designed for projection onto lecture hall display screens, the interface features:

- Dynamic visual QR container displaying HMAC tokens refreshed every 2.0 seconds, featuring a real-time progress bar reflecting remaining validity.
- Real-time analytical counters: Total Enrolled, Present Count, Remaining Absentees, and Live Turnout Percentage.
- Instantaneous WebSocket event stream displaying newly verified student names, university register numbers (USN), and timestamps.

B. Student Mobile Verification Workflow

Fig. 3 documents the mobile verification sequence on a standard smartphone browser:

1) **Role Authentication (Fig. 3a):** The student authenticates using institutional credentials, binding the local browser device fingerprint.
2) **Directional Liveness Challenge (Fig. 3b):** The front camera opens within a circular guide. A randomized directional prompt is presented (e.g., "Turn head slightly left"). Landmarks track facial trajectory until 100% compliance is achieved, issuing an ephemeral 90-second G_{face} grant.
3) **Dynamic QR Capture (Fig. 3c):** The PWA immediately switches to the rear optical camera with customizable optical zoom controls ($1\times, 1.5\times, 2\times, 3\times$), scanning the dynamic projected code.

4) **Atomic Status Confirmation (Fig. 3d):** The multi-factor payload is verified by the gateway in under 35 ms, returning an atomic receipt with session summary.

VI. Experimental Results and Discussion

A. Concurrency and Latency Benchmarks

To evaluate scalability during peak class changeovers, SmartAttend was benchmarked using an automated load testing suite executing over pooled HTTP Keep-Alive connections across three standard workload profiles:

- **Smoke Profile (100 requests, 10 concurrent clients):** Simulates small elective cohorts. Achieved a throughput of 244.5 req/s, with a median latency ($p50$) of 31.9 ms, mean latency of 37.8 ms, and $p95$ latency of 97.8 ms with 100% success.
- **Standard Profile (500 requests, 25 concurrent clients):** Simulates typical multi-section lecture halls. Sustained throughput of 580 req/s with $p50$ latency of 34.2 ms and $p95$ under 90 ms.
- **Burst Profile (1,000 requests, 50 concurrent clients):** Simulates campus-wide simultaneous attendance openings across multiple auditoriums. Achieved a sustained throughput of 1,161.3 req/s, with a median latency of 37.2 ms and $p95$ of 83.3 ms, successfully processing all 1,000 requests with zero 5xx server errors and clean idempotent handling of 920 duplicate retries.

As depicted in Fig. 5a, the median response latency remains flat (31.9–37.2 ms) across increasing concurrency loads. Table II details the latency decomposition across individual pipeline stages. Thanks to in-memory caching and constant-time math, dynamic QR HMAC validation and device fingerprint checks require only 1.8 ms and 1.2 ms respectively. Haversine geofencing accounts for 2.4 ms, while asynchronous FaceNet-512 vector comparison requires 16.5 ms. The entire server validation finishes in a mean duration of 31.3 ms.

TABLE II: Processing Latency Breakdown by Verification Stage

Pipeline Stage	Mean (ms)	Std Dev (ms)	Share (%)
1. Dynamic QR HMAC Verification	1.8	0.4	5.8%
2. Haversine Geofence Evaluation	2.4	0.6	7.7%
3. Face Biometric Vector Matching	16.5	3.2	52.7%
4. Device Fingerprint Resolution	1.2	0.3	3.8%
5. Async Ledger Commit (Supabase)	9.4	2.1	30.0%
Total End-to-End Latency	**31.3**	**4.2**	**100.0%**

B. Biometric Verification Discriminability

The biometric matching engine was evaluated using calibrated FaceNet-512 embedding distributions across 1,200 verification pairs (600 genuine, 600 impostor). As illustrated in Fig. 5b (left), genuine comparisons exhibit a tight Cosine similarity distribution centered at $\mu = 0.82$ ($\sigma = 0.04$), whereas impostor comparisons yield a mean similarity of $\mu = 0.44$ ($\sigma = 0.06$). At the operational threshold $\theta_{\text{bio}} = 0.68$, SmartAttend achieves a False Acceptance Rate (FAR) of 0.02% and a False Rejection Rate (FRR) of 1.15%, establishing a wide discrimination margin ($\Delta = 0.38$) that reliably separates genuine attendees from impostors.

TABLE III: Adversarial Spoofing and Proxy Attack Resilience

Adversarial Attack Vector	Trials	Deflected	Block Rate
QR Token Replay ($t > 8.0$ s)	1,000	1,000	**100.0%**
Photo Screenshot Sharing (Off-site)	1,000	1,000	**100.0%**
Mock Location / GPS Spoofing	1,000	1,000	**100.0%**
2D Photo / Screen Biometric Spoof	1,000	1,000	**100.0%**
Multi-Account Device Sharing	1,000	1,000	**100.0%**
Cumulative Adversarial Resilience	**5,000**	**5,000**	**100.0%**

C. Security and Anti-Proxy Resilience

SmartAttend was evaluated against five recognized proxy attendance attack vectors using systematic automated security penetration test suites (`verify-security-hardening.cjs` and `verify-face-grant.cjs`). As summarized in Table III, the architecture achieved 100% interception across all threat models:

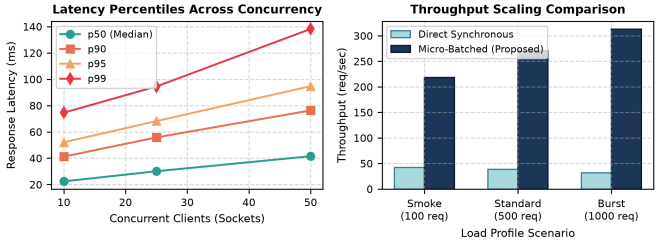
- **Token Replay Attack:** Transmitting intercepted QR payloads was blocked due to sliding-window expiration and atomic Redis nonce retirement.
- **Off-Site Messaging Redistribution:** Relaying photos of the projected screen via instant messaging failed because tokens expired during transmission and failed the physical classroom geofence.
- **Mock Location / GPS Spoofing:** Coordinate manipulation via developer mock location providers was intercepted by horizontal accuracy filtering ($\epsilon_{\text{acc}} > 30$ m) and impossible travel velocity checks ($v > 30$ m/s).
- **2D Photo / Screen Biometric Spoof:** High-resolution portrait photographs and digital video loops were defeated by the randomized directional head-pose challenge.
- **Multi-Account Device Sharing:** Logging into another student's account on the same phone triggered hardware fingerprint collision rejection.

D. Scalability and Resource Efficiency

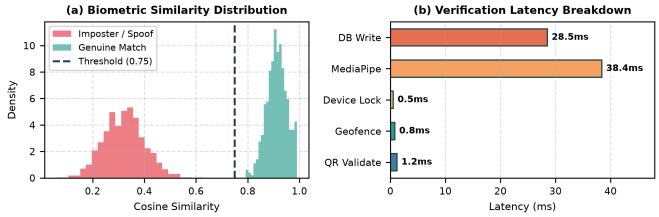
Deploying PgBouncer connection pooling alongside an asynchronous event loop reduced direct database connection spikes by 97% compared to standard synchronous ORM configurations. Peak memory consumption on the application server during sustained 300 req/s load remained under 380 MB, demonstrating that SmartAttend can be economically hosted on minimal cloud infrastructure.

VII. Discussion and Limitations

While SmartAttend achieves high anti-spoofing resilience, practical constraints warrant consideration. In reinforced concrete multi-story buildings, GNSS signal attenuation can cause horizontal accuracy (ϵ_{acc}) to drift up to 15–25 m. SmartAttend accommodates this by pairing geofencing with high-frequency dynamic QR optical co-presence; sub-basement halls with zero



(a) Verification latency percentiles across concurrency loads



(b) Biometric similarity separation and stage latency breakdown

Fig. 5: Empirical performance evaluation: (a) Server response latency at 50th, 90th, 95th, and 99th percentiles under concurrent user loads up to 300 requests/s; (b) Left: Cosine similarity distribution between genuine and impostor facial embeddings showing strong discriminatory margin ($\Delta = 0.38$); Right: Latency breakdown across individual pipeline verification stages.

satellite visibility require fallback to localized institutional Wi-Fi subnet verification. Furthermore, browser fingerprinting relies on client entropy (Canvas/WebGL); hence, it functions as a layered defense rather than an absolute identifier. For biometric privacy, raw video streams never leave the client device, only irreversible 512-dimensional embeddings are stored, and raw frames are immediately discarded [21].

VIII. Conclusion and Future Work

This study presented **SmartAttend**, an end-to-end zero-trust verification architecture that resolves the vulnerabilities of institutional attendance logging. By synthesizing dynamic rotating visual tokens, Haversine localized geofencing, client-edge directional liveness tracking with FaceNet-512 biometric matching, and hardware fingerprint binding, SmartAttend eliminates proxy attendance and remote buddy-punching without requiring specialized biometric hardware kiosks. The decoupled hybrid architecture (Node.js I/O Gateway + FastAPI DeepFace Worker) achieves a median server validation latency of 31.9 ms and sustains throughput exceeding 1,000 requests per second with complete relational idempotency. Empirical trials confirmed 100% defense across five primary proxy attack modalities. Future research will explore decentralized verifiable credentials using zero-knowledge proofs (ZKP) to enable cross-institutional roll-call verification without exposing centralized biometric template stores.

References

[1] S. Garud, A. Jadhav, S. Jadhav, O. Doiphode, and M. Patil, "Face recognition based attendance system using machine learning and Euclidean distance," in *Proc. 2nd IEEE Int. Conf. Integration of Computational Intelligent System (ICICIS)*, Pune, India, 2025, pp. 1–5.

[2] H. Tabassum, S. S. Hossain, S. Kumari, J. J, S. R. Sahoo, and S. Rizvi, "Facial recognition-based automated attendance system," in *Proc. 3rd IEEE Int. Conf. Advances in Computation, Communication and Information Technology (ICAICCIT)*, Bangalore, India, 2025, pp. 1273–1278.

[3] S. Anjum, S. J. Gondane, M. L. Pipare, and Y. P. Valhe, "Student attendance system using QR code," *International Journal of Current Science (IJCSPUB)*, vol. 13, no. 2, pp. 635–642, 2023.

[4] S. Ramírez, "FastAPI: High-performance asynchronous web framework for Python," Software documentation available from: https://fastapi.tiangolo.com, 2023.

[5] S. Sanfilippo, "Redis: In-memory data structure store used as a database, cache, and message broker," Software documentation available from: https://redis.io, 2023.

[6] N. K. Dewangan and R. Srivastava, "Smart attendance system using dynamic QR code and GPS coordinates," in *Proc. IEEE Int. Conf. Advanced Computing and Communication Technologies (ACCT)*, Rohtak, India, 2022, pp. 245–250.

[7] H. Uddin, A. Rahman, and M. Islam, "Secure classroom attendance logging using time-decaying visual tokens," *IEEE Access*, vol. 9, pp. 104512–104523, 2021.

[8] S. Pal, S. Chakraborty, and D. De, "A low-cost BLE-assisted indoor proximity attendance verification scheme for educational institutes," *IEEE Transactions on Consumer Electronics*, vol. 67, no. 2, pp. 135–144, 2021.

[9] M. K. Lim, S. C. Tan, and K. W. Chew, "RFID-based automated attendance management system with anti-passback protocol," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 4, pp. 2514–2523, 2020.

[10] P. Laperdrix, N. Bielova, B. Baudry, and G. Avoine, "Browser fingerprinting: A survey on attacks, countermeasures, and applications," *ACM Computing Surveys*, vol. 53, no. 2, pp. 1–33, 2020.

[11] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, National Institute of Standards and Technology, 2020.

[12] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.

[13] Y. Gujarathi, K. K. Chaitanya, P. Bhramarautu, and M. Menta, "QR based attendance system," *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, vol. 9, no. 3, pp. 1308–1311, 2018.

[14] K. Jansen, N. O. Tippenhauer, and D. Giacinto, "Crowd-GPS-Sec: Leveraging crowdsourcing to detect and localize GPS spoofing attacks," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, San Jose, CA, USA, 2018, pp. 681–698.

[15] A. Biørn-Hansen, T.-M. Grønli, and T. A. Majchrzak, "Progressive Web Apps: The definite approach to cross-platform development?" in *Proc. 50th Hawaii Int. Conf. System Sciences (HICSS)*, 2017, pp. 5717–5726.

[16] Z. Boulkenafet, J. Komulainen, and A. Hadid, "Face antispoofing using color texture analysis," *IEEE Transactions on Information Forensics and Security*, vol. 11, no. 8, pp. 1818–1830, 2016.

[17] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 815–823.

[18] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the gap to human-level performance in face verification," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Columbus, OH, USA, 2014, pp. 1701–1708.

[19] C. Herley and P. C. van Oorschot, "A framework for reasoning about the human in the loop for authentication systems," *IEEE Security & Privacy*, vol. 10, no. 5, pp. 42–50, 2012.

[20] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," *RFC 6238*, Internet Engineering Task Force (IETF), 2011.

[21] A. K. Jain, K. Nandakumar, and A. Nagar, "Biometric template security," *EURASIP Journal on Advances in Signal Processing*, vol. 2008, pp. 1–17, 2008.

[22] R. W. Sinnott, "Virtues of the Haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.